Response. Xiyao Long

Apache Hadoop YARN: Yet another resource negotiator

The paper introduces a new resource scheduling framework YARN. The motivation of this system is that although Hadoop can leverage the parallel computing power by executing the mapping tasks on different data blocks, its current model is still not ideal for some tasks such as machine learning. Machine learning tasks can need several iterations of computations on the entire datasets before converging the results into one, Hadoop would possibly do scheduling for each iteration depending on how the data blocking needs to be done.

YARN manage to separate the programming model and resource management functions while let the resource management protocols to take better account of locality awareness, utilization compared to traditional Hadoop. In this infrastructure, there are several roles including Resource Manager(**RM**), central Allocator(overall), Application Master(**AM**) and Node Master(**NM**) take charge of communication and coordination between physical machines in the computing grid.

AM and RM are the central part of YARN. AM request resource for a specific job by sending messages to the RM and wait the RM to coordinate its request along many of other such requests. It redeems the allocated resources corresponding to its request by keep sending heartbeat message to RM to find out the allocation status. For a MapReduce job, an AM can correspond a mapper or reducer task to a container in the allocated resource. As YARN allows finer granularity in RequestResource Message it can specify preferred locality and has a mapper task to be near the place where the data block is hosted. The YARN system is made very aware of the locality preference, on top of this, protocols implemented in YARN can enforce the system to prioritize capacity or fairness in the system.

Another property of the system is that the actual resource granted may be different from what is requested by AM. The AM executes physical plan based on the actual resource allocated not the requested resource. This adaptation possibly makes the system has better utilization of the resource. The authors did experiment to demonstrate the YARN system can handle large # of applications and large jobs use more resources compared to smaller jobs. This performance advantage allows the production of Yahoo! In YARN beneficial.

They further talked about how YARN handles RM failure or restart, in the paper they briefly discussed a strategy which the RM would reinitiate from previously saved machine and the running AMs should finish their work and resync with the RM. Although it's not clear such solution has been successfully implemented.

One confusion I have is that they did not make it explicit is why YARN can avoid the rescheduling problem they talked earlier in the machine learning case. By simply guessing, it can be that the AM now utilize resources dynamically as it can constantly update the RM about its resource usage, the system doesn't need to repeat the scheduling as much as it previously does.